MultiAgent OS

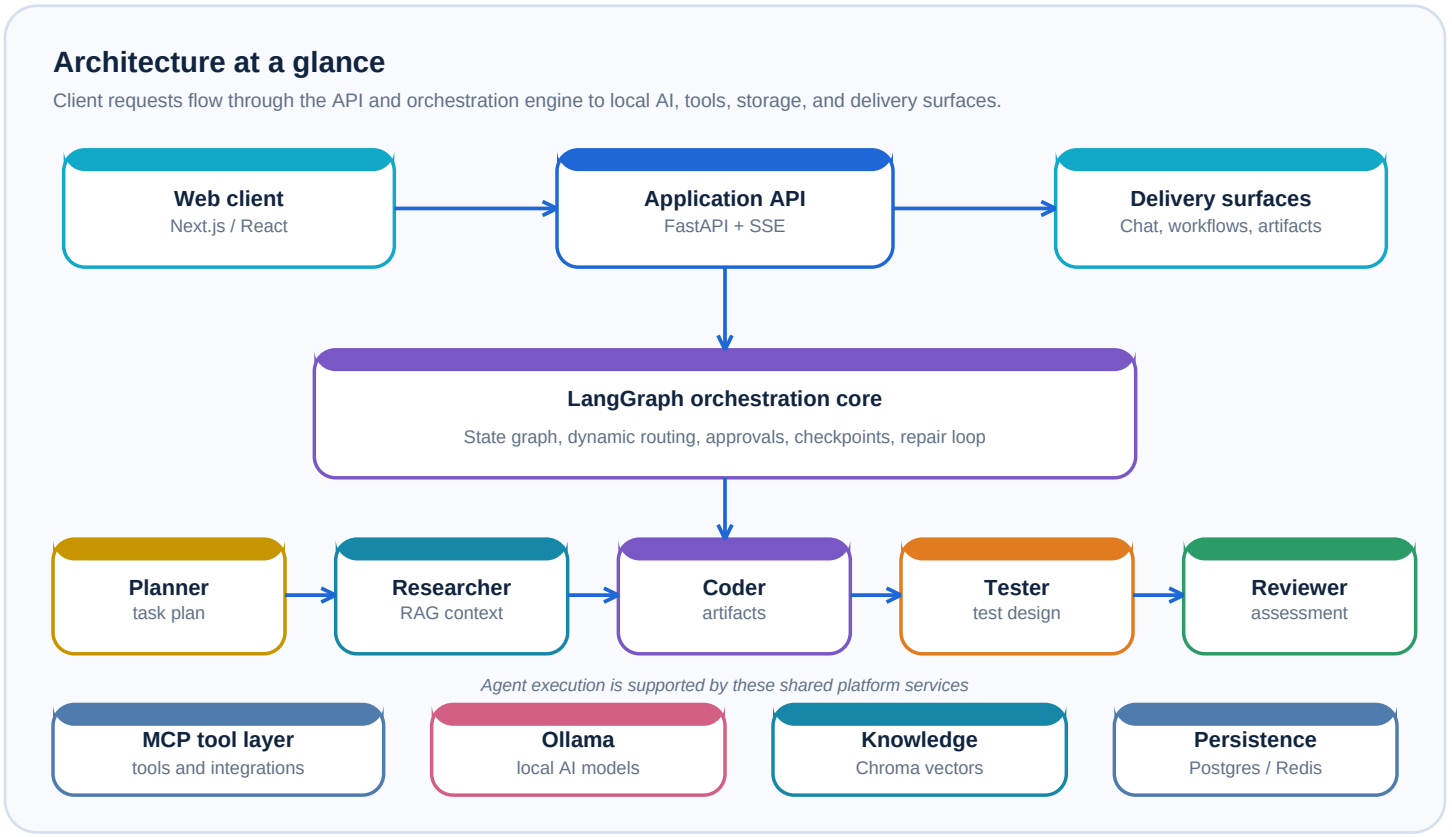
Overall project summary and architecture

Purpose

MultiAgent OS is a full-stack AI orchestration platform for taking a user request from conversation to a coordinated, multi-step software-workflow outcome. It combines a web application, an asynchronous API, local large-language-model inference, a LangGraph execution engine, knowledge retrieval, tool integrations, isolated workspaces, and persistent operational data.

The product supports two interaction patterns: concise assistant-style exchanges and richer workflow runs that coordinate specialist agents. During workflow runs, the platform can stream events to the browser, preserve execution context, route work between agents, create artifacts, and expose the resulting workspace through dedicated interfaces.

System architecture



The browser-facing Next.js application presents dashboards, chat, workflow management, workspaces, knowledge, tools, analytics, governance, and operations views. Its service modules call FastAPI endpoints, while Server-Sent Events carry incremental workflow activity back to the UI.

Core execution flow

Stage	Responsibility	Primary output
Planner	Interprets the request, creates an execution plan, and determines the agent sequence.	Plan and required-agent list
Researcher	Builds context from the knowledge layer and approved tools.	Research notes
Coder	Creates or revises source artifacts in an isolated workspace.	Generated code and files
Tester	Produces tests and coverage-oriented analysis for generated work.	Test artifacts and analysis
Quality gate	Runs deterministic validation over the workspace, including syntax/import checks, Ruff, pytest, and Bandit.	Validation evidence
Reviewer	Uses the workflow context and validation results to provide a qualitative assessment.	Review output

Major project components

Frontend application

Next.js 16, React 19, TypeScript, TanStack React Query, Axios, Tailwind CSS, Three.js, and Lucide. The App Router organizes public pages and the authenticated application shell; components and hooks support chat, system diagnostics, workflow visualisation, navigation, and status-aware UX.

Backend API and services

FastAPI provides REST and streaming interfaces. Route modules cover chat, agents, workflows, workflow builder, workspaces, artifacts, knowledge, tools, analytics, operations, tenants, authentication, prompts, health, and AIOps. Service and repository layers keep business rules, storage access, scheduling, and artifact management separate from endpoint handlers.

Orchestration and agents

LangGraph supplies a shared-state execution graph. The planner, researcher, coder, tester, and reviewer are specialized agent modules. Dynamic routing can reduce or expand the selected path; for build-oriented work it enforces the core sequence. A failed validation result can feed a bounded repair loop back to the coder.

AI and knowledge

Ollama is the local model provider. The knowledge pipeline accepts documents, chunks content, generates embeddings, stores vectors in ChromaDB, and retrieves relevant context for planning and research. Planning memory provides an additional vector-backed context surface.

Tools and workspaces

The Model Context Protocol implementation registers filesystem, PostgreSQL, GitHub, HTTP, and terminal tools. Workspace services isolate generated files by session, track metadata, and support artifact extraction and delivery.

Persistence and background work

PostgreSQL is accessed with asynchronous SQLAlchemy and migrated with Alembic. Redis serves Celery broker/result-backend needs; workers and scheduled jobs separate long-running workflow, indexing, analytics, and tool work from interactive API handling.

Security and governance

Authentication, rate limiting, structured exception handling, DLP masking, and prompt-security checks live in dedicated backend modules. The application also includes tenant, workspace, operations, analytics, and AIOps domains for governed use of the platform.

Deployment topology

The repository includes Docker Compose definitions for PostgreSQL, Redis, the FastAPI backend, Celery worker, Celery Beat scheduler, the Next.js frontend, and Nginx. The backend reaches Ollama through a configurable base URL, supporting a local model runtime while the remainder of the application runs as coordinated services.

Repository map

Area	What it contains
backend/app	FastAPI application, agents, orchestration graph, API routes, services, repositories, schemas, models, security, knowledge, MCP server, and workers.
backend/tests	Backend tests covering workflow behavior, quality-gate rules, workspace handling, routing, authentication, embeddings, chat, and artifact extraction.
frontend/app	Next.js routes for public pages, dashboard, chat, workflows, workspace, knowledge, tools, analytics, governance, settings, and AIOps.
frontend/components, hooks, services	Reusable UI, client-side interaction logic, and typed API service modules.
docker-compose.yml and nginx	Containerized service topology and reverse-proxy configuration.

Operating model

A request enters through the web client and is handled by the FastAPI application. For a workflow request, the orchestration engine carries a shared state object through the selected agents, records checkpoints, and streams progress to the browser. The application can pause at configured approval points and resume from the persisted workflow context.

Generated work is placed in a session-scoped workspace. The deterministic quality gate reads that workspace and returns validation evidence to the graph; the reviewer receives the generated output, testing context, research notes, and validation results. Persistent relational records support chat, workflows, agent executions, artifacts, knowledge metadata, tenants, and metrics.

Service boundaries

Interactive API traffic, background jobs, vector retrieval, model inference, file workspaces, and database persistence are kept in separate modules and services. This separation lets the UI stay responsive while workflows, indexing, analytics, and tool execution operate through their respective workers and integrations.